

**VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE**

Faculty of Information Technology



**REPORT
LAB 1: HDFS**

COURSE: INTRODUCTION TO BIG DATA

LECTURERS:

- Nguyen Ngoc Thao
- Huynh Lam Hai Dang
- Tran Huy Ban

CLASS: 23KHDL2

STUDENT INFORMATION:

- Phan Nhat Thien Nhan – Student ID: 23127096

Ho Chi Minh City, July 11, 2026.

Contents

1	HADOOP PSEUDO-DISTRIBUTED CLUSTER SETUP (SINGLE NODE)	4
1.1	Inspecting the WSL Environment and Creating the Hadoop User	4
1.1.1	Step 1: Inspect the operating system and current account	4
1.1.2	Step 2: Create a Hadoop account for this lab	4
1.1.3	Step 3: Verify the Hadoop account	5
1.2	Java Environment and Required Packages	6
1.2.1	Step 1: Update Ubuntu and install dependencies	6
1.2.2	Step 2: Verify OpenJDK 17	6
1.3	Configuring Passwordless SSH	7
1.3.1	Step 1: Start SSH and test the initial connection	7
1.3.2	Step 2: Generate and authorize an RSA key pair	7
1.3.3	Step 3: Verify passwordless SSH	7
1.4	Downloading, Installing, and Configuring Apache Hadoop	8
1.4.1	Step 1: Download Apache Hadoop 3.5.0 binary	8
1.4.2	Step 2: Extract and organize the Hadoop directory	8
1.4.3	Step 3: Configure environment variables in <code>.bashrc</code>	9
1.4.4	Step 4: Activate and verify the environment	10
1.5	Configuring the Hadoop Environment and XML Files	10
1.5.1	Step 2: Configure <code>core-site.xml</code>	11
1.5.2	Step 1: Set the Java path in <code>hadoop-env.sh</code>	11
1.5.3	Step 3: Configure <code>hdfs-site.xml</code>	12
1.5.4	Step 4: Configure <code>mapred-site.xml</code>	13
1.5.5	Step 5: Configure <code>yarn-site.xml</code>	14
1.6	Formatting and Starting the Hadoop Cluster	15
1.6.1	Step 1: Format the NameNode	15
1.6.2	Step 2: Start HDFS and verify its processes	15
1.7	Cluster Status Verification via Web UI	16
1.7.1	Step 1: Explore the NameNode Web UI	16
1.7.2	Step 2: Start YARN	16
1.7.3	Step 3: Explore the ResourceManager Web UI	16
2	Pseudo-Distributed Mode	17
2.1	Step 1: Creating the System Directory and Dedicated User	17
2.2	Step 2: Managing the Student Directory, Uploading Data, and Configuring Permissions	17
2.3	Step 3: Executing the JAR Program for Automated Verification	18
3	Word Count	19
3.1	Step 1: Preparing the Local Workspace	19
3.2	Step 2: Creating the HDFS Input Directory and Uploading the Dataset	19
3.3	Step 3: Reading the Uploaded Dataset	19
3.4	Step 4: Installing Scala and Creating the Source File	20
3.5	MapReduce Algorithm	21
3.5.1	Map Function	21
3.5.2	Reduce Function	22
3.5.3	Main Function	22
3.5.4	MapReduce Data Flow	23

3.6	Step 5: Compiling the Scala Application	24
3.7	Step 6: Packaging the JAR and Resolving Command Errors	24
3.8	Step 7: Submitting the MapReduce Job to YARN	25
3.9	Step 8: Exporting and Verifying the TSV Result	26

1 HADOOP PSEUDO-DISTRIBUTED CLUSTER SETUP (SINGLE NODE)

Prerequisites: Follow the steps mentioned in the report below to first enable Windows Subsystem for Linux and then install a Linux distribution of your choice.

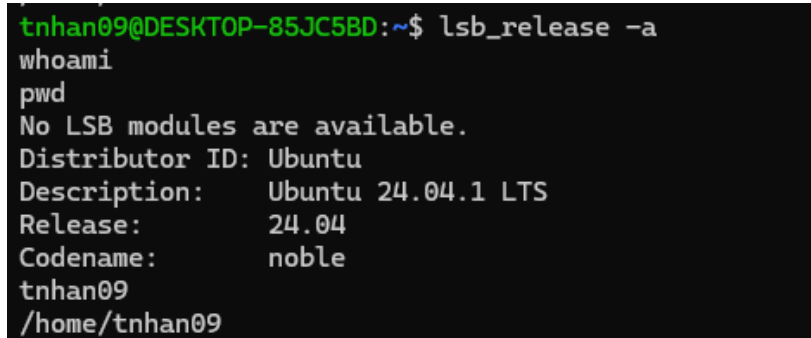
Reference: The Hadoop installation procedure was adapted from the tutorial [“Hadoop Installation on Windows 10 using WSL”](#).

1.1 Inspecting the WSL Environment and Creating the Hadoop User

1.1.1 Step 1: Inspect the operating system and current account

Before installing Hadoop, the Ubuntu release, current user, and home directory were checked:

```
lsb_release -a
whoami
pwd
```



```
tnhan09@DESKTOP-85JC5BD:~$ lsb_release -a
whoami
pwd
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.1 LTS
Release:        24.04
Codename:       noble
tnhan09
/home/tnhan09
```

Figure 1: Ubuntu 24.04.1 LTS, the initial WSL account, and its home directory.

1.1.2 Step 2: Create a Hadoop account for this lab

Now we create a separate Linux account named `hadoop` and add to the `sudo` group:

```
sudo adduser hadoop
sudo usermod -aG sudo hadoop
```

```
sudo: add: Command not found
tnhan09@DESKTOP-85JC5BD:~$ sudo adduser hadoop
info: Adding user `hadoop' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hadoop' (1002) ...
info: Adding new user `hadoop' (1002) with group `hadoop (1002)' ...
info: Creating home directory `/home/hadoop' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for hadoop
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] Y
info: Adding new user `hadoop' to supplemental / extra groups `users' ...
info: Adding user `hadoop' to group `users' ...
tnhan09@DESKTOP-85JC5BD:~$ sudo usermod -aG sudo hadoop
tnhan09@DESKTOP-85JC5BD:~$ |
```

Figure 2: Creating the hadoop account and granting administrative privileges.

The session was then switched to the new account:

```
sudo su - hadoop
```

```
tnhan09@DESKTOP-85JC5BD:~$ sudo su - hadoop
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Jul  7 19:37:29 +07 2026

System load:  0.01               Processes:    33
Usage of /:   0.2% of 1006.85GB   Users logged in: 1
Memory usage: 2%                IPv4 address for eth0: 172.17.63.119
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/hadoop/.hushlogin file.
hadoop@DESKTOP-85JC5BD:~$ |
```

Figure 3: Logging in to the dedicated hadoop account.

1.1.3 Step 3: Verify the Hadoop account

```
whoami
pwd
groups
```

```
hadoop@DESKTOP-85JC5BD:~$ whoami
pwd
groups
hadoop
/home/hadoop
hadoop sudo users
```

Figure 4: Verifying the username, home directory, and group memberships.

- **Result:** The working account is `hadoop`, its home directory is `/home/hadoop`, and it belongs to the `hadoop`, `sudo`, and `users` groups.

1.2 Java Environment and Required Packages

1.2.1 Step 1: Update Ubuntu and install dependencies

We update `apt` and then installed needed packages for the following steps of installing Hadoop.

```
sudo apt update
sudo apt upgrade -y
sudo apt install openjdk-17-jdk openssh-server rsync wget nano -y
```

```
hadoop@DESKTOP-85JC5BD:~$ sudo apt update
sudo apt upgrade -y
sudo apt install openjdk-17-jdk openssh-server rsync wget nano -y
[sudo] password for hadoop:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
70 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
Get another security update through Ubuntu Pro with 'esm-apps' enabled:
  kubectx
Learn more about Ubuntu Pro at https://ubuntu.com/pro
The following NEW packages will be installed:
  ethtool
The following packages will be upgraded:
  apparmor apt apt-utils base-files binutils binutils-common binutils-x86-64-linux-gnu cloud-init coreutils
  dconf-gsettings-backend dconf-service dhcpcd-base dmsetup gtk-update-icon-cache iproute2 landscape-client
  landscape-common libapparmor1 libapt-pkg6.0t64 libattr1 libbinutils libbsd0 libcryptsetup12 libctf-nobfd0 libctf0
  libdconf1 libdevmapper1.02.1 libgmp10 libgpg-error-l10n libgpg-error0 libgprofng0 libgtk-3-0t64 libgtk-3-bin
  libgtk-3-common libidn2-0 libldap-common libldap2 libmd0 libmpfr6 libnetplan1 libpciaccess0 libpcre2-8-0 libselinux1
  libsfml1 libunistring5 libunwind8 libxmlb2 lshw motd-news-config netplan-generator netplan.io pci.ids
```

Figure 5: Updating Ubuntu and installing Java, SSH, and supporting utilities.

1.2.2 Step 2: Verify OpenJDK 17

```
java -version
```

```
hadoop@DESKTOP-85JC5BD:~$ java -version
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment (build 17.0.19+10-1-24.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.19+10-1-24.04.2-Ubuntu, mixed mode, sharing)
```

Figure 6: OpenJDK 17 runtime verification.

- **Result:** The command returned OpenJDK **17.0.19**, confirming that the Java runtime required by Hadoop 3.5.0 was available.

1.3 Configuring Passwordless SSH

Hadoop startup scripts use SSH even in pseudo-distributed mode. Therefore, the local SSH service and public-key authentication were configured.

1.3.1 Step 1: Start SSH and test the initial connection

```
sudo service ssh start
ssh localhost
```

```
hadoop@DESKTOP-85JCSBD:~$ sudo service ssh start
[sudo] password for hadoop:
hadoop@DESKTOP-85JCSBD:~$ ssh localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:T/4yCdmnbviz2If+7brqRK2/wDnUtrcQeX5Y0t+0tII..
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
hadoop@localhost's password:
```

Figure 7: The first SSH connection accepted the host key but still requested a password.

The password prompt showed that passwordless authentication had not yet been configured.

1.3.2 Step 2: Generate and authorize an RSA key pair

If you cannot ssh to localhost without a passphrase, run the following command to initialize your private and public keys:

```
sudo service ssh start
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys
```

```
hadoop@DESKTOP-85JC5BD:~$ sudo service ssh start
hadoop@DESKTOP-85JC5BD:~$ ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys
Generating public/private rsa key pair.
Your identification has been saved in /home/hadoop/.ssh/id_rsa
Your public key has been saved in /home/hadoop/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:Unbh+NzckKezra+X0dEUgp3pyAZxLyUN0oMOXVBAS+0 hadoop@DESKTOP-85JC5BD
The key's randomart image is:
+---[RSA 3072]-----+
|
|  .+=+oo.o o
|  . o=.o.* .
|  .=.+=.o .
|  .o+Eo*o* .
|  ..+S=+o= .o
|  o.. o +. .
|  . . .+
|  . =
|  o+..
+-----[SHA256]-----+
```

Figure 8: Generating the RSA key pair and configuring authorized keys.

The mode 600 restricts `authorized_keys` to owner read/write access, which satisfies SSH security.

1.3.3 Step 3: Verify passwordless SSH

Make sure you can SSH to localhost in Ubuntu:

```
ssh localhost
```

```

+-----[SHA256]-----+
hadoop@DESKTOP-85JC5BD:~$ ssh localhost
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Jul  7 20:04:42 +07 2026

System load:  0.01               Processes:            36
Usage of /:   0.2% of 1006.85GB   Users logged in:     1
Memory usage: 2%                IPv4 address for eth0: 172.17.63.119
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

```

Figure 9: Successful passwordless SSH login to localhost.

- **Result:** The second connection entered the local Hadoop account without requesting the account password.

1.4 Downloading, Installing, and Configuring Apache Hadoop

1.4.1 Step 1: Download Apache Hadoop 3.5.0 binary

Run the following command to download the latest binary for Hadoop 3.5.0:

```
wget https://dlcdn.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
```

```

hadoop@DESKTOP-85JC5BD:~$ wget https://dlcdn.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
--2026-07-07 21:18:51-- https://dlcdn.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 581280703 (554M) [application/x-gzip]
Saving to: 'hadoop-3.5.0.tar.gz'

hadoop-3.5.0.tar.gz      100%[=====>] 554.35M  8.73MB/s   in 76s
2026-07-07 21:20:02 (7.27 MB/s) - 'hadoop-3.5.0.tar.gz' saved [581280703/581280703]

```

Figure 10: Downloading the Hadoop 3.5.0 binary archive from Apache.

1.4.2 Step 2: Extract and organize the Hadoop directory

Now we create a `/hadoop` folder under the user home folder and then extract the binary package from the previous step into this folder

```

mkdir ~/hadoop
tar -xzf hadoop-3.5.0.tar.gz
mv hadoop-3.5.0 hadoop
ls -l ~
ls -l ~/hadoop

```



```

hadoop@DESKTOP-85JC5BD:~$ tar -xzf hadoop-3.5.0.tar.gz
hadoop@DESKTOP-85JC5BD:~$ mv hadoop-3.5.0 hadoop
hadoop@DESKTOP-85JC5BD:~$ ls -l ~
ls -l ~/hadoop
total 567668
drwxr-xr-x 11 hadoop hadoop      4096 Mar 25 00:31 hadoop
-rw-rw-r-- 1 hadoop hadoop 581280703 Apr  2 03:43 hadoop-3.5.0.tar.gz
total 124
-rw-r--r-- 1 hadoop hadoop   933 Mar 25 00:31 Dockerfile
-rw-rw-r-- 1 hadoop hadoop 29329 Mar 24 23:47 LICENSE-binary
-rw-rw-r-- 1 hadoop hadoop 15886 Feb 24 07:10 LICENSE.txt
-rw-rw-r-- 1 hadoop hadoop 27373 Feb 24 07:10 NOTICE-binary
-rw-rw-r-- 1 hadoop hadoop  1541 Feb 24 07:10 NOTICE.txt
-rw-rw-r-- 1 hadoop hadoop   175 Feb 24 07:10 README.txt
drwxr-xr-x 2 hadoop hadoop   4096 Mar 25 00:31 bin
drwxr-xr-x 3 hadoop hadoop   4096 Mar 25 00:31 compose
drwxr-xr-x 3 hadoop hadoop   4096 Mar 24 23:57 etc
drwxr-xr-x 2 hadoop hadoop   4096 Mar 25 00:31 include
drwxr-xr-x 3 hadoop hadoop   4096 Mar 25 00:31 lib
drwxr-xr-x 4 hadoop hadoop   4096 Mar 25 00:31 libexec
drwxr-xr-x 2 hadoop hadoop   4096 Mar 25 00:31 licenses-binary
drwxr-xr-x 3 hadoop hadoop   4096 Mar 24 23:57 sbin
drwxr-xr-x 4 hadoop hadoop   4096 Mar 25 00:43 share
hadoop@DESKTOP-85JC5BD:~$ |

```

Figure 11: Extracting Hadoop and verifying the /home/hadoop/hadoop directory.

- **Result:** The Hadoop installation is located at /home/hadoop/hadoop and is owned by the hadoop account.

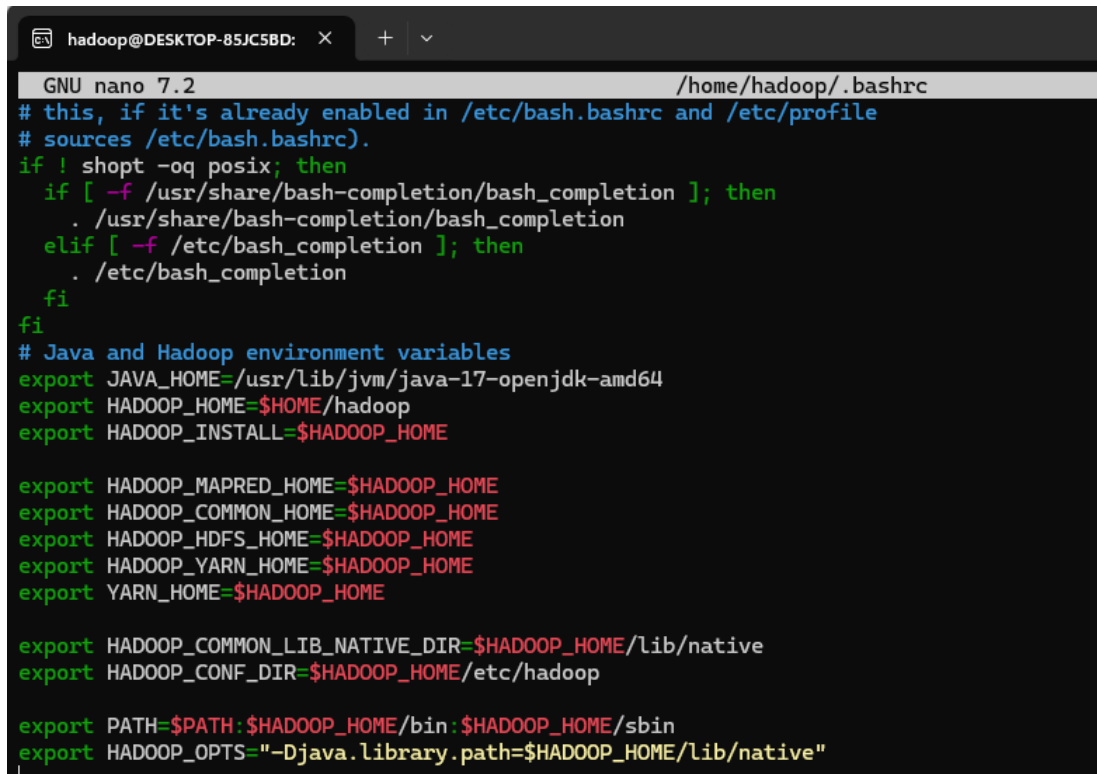
1.4.3 Step 3: Configure environment variables in .bashrc

First, we open .bashrc using `nano ~/.bashrc` and then the following variables were added to the end of ~/.bashrc:

```

export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=$HOME/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"

```



```

GNU nano 7.2 /home/hadoop/.bashrc
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi
# Java and Hadoop environment variables
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=$HOME/hadoop
export HADOOP_INSTALL=$HADOOP_HOME

export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME

export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop

export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"

```

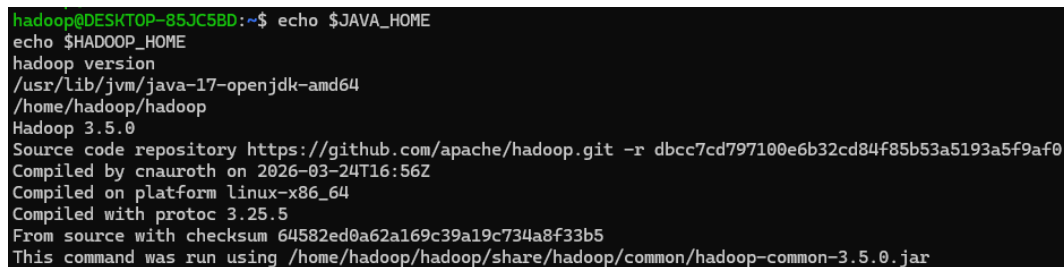
Figure 12: Hadoop and Java environment variables appended to `.bashrc`.

1.4.4 Step 4: Activate and verify the environment

```

source ~/.bashrc
echo $JAVA_HOME
echo $HADOOP_HOME
hadoop version

```



```

hadoop@DESKTOP-85JC58D:~$ echo $JAVA_HOME
/usr/lib/jvm/java-17-openjdk-amd64
hadoop@DESKTOP-85JC58D:~$ echo $HADOOP_HOME
/home/hadoop/hadoop
hadoop@DESKTOP-85JC58D:~$ hadoop version
Hadoop 3.5.0
Source code repository https://github.com/apache/hadoop.git -r dbcc7cd797100e6b32cd84f85b53a5193a5f9af0
Compiled by cnauroth on 2026-03-24T16:56Z
Compiled on platform linux-x86_64
Compiled with protoc 3.25.5
From source with checksum 64582ed0a62a169c39a19c734a8f33b5
This command was run using /home/hadoop/hadoop/share/hadoop/common/hadoop-common-3.5.0.jar

```

Figure 13: Verifying `JAVA_HOME`, `HADOOP_HOME`, and Hadoop 3.5.0.

- **Result:** Hadoop commands resolve correctly and the active version is Hadoop **3.5.0**.

1.5 Configuring the Hadoop Environment and XML Files

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

```
GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/hadoop-env.sh *
#
# When building Hadoop, one can add the class paths to the commands
# via this special env var:
# export HADOOP_ENABLE_BUILD_PATHS="true"
#
# To prevent accidents, shell commands be (superficially) locked
# to only allow certain users to execute certain subcommands.
# It uses the format of (command)_(subcommand)_USER.
#
# For example, to limit who can execute the namenode command,
# export HDFS_NAMENODE_USER=hdfs
#
###
# Registry DNS specific parameters
###
# For privileged registry DNS, user to run as after dropping privileges
# This will replace the hadoop.id.str Java property in secure mode.
# export HADOOP_REGISTRYDNS_SECURE_USER=yarn
#
# Supplemental options for privileged registry DNS
# By default, Hadoop uses jsvc which needs to know to launch a
# server jvm.
# export HADOOP_REGISTRYDNS_SECURE_EXTRA_OPTS="-jvm server"
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

Figure 14: Setting JAVA_HOME in hadoop-env.sh.

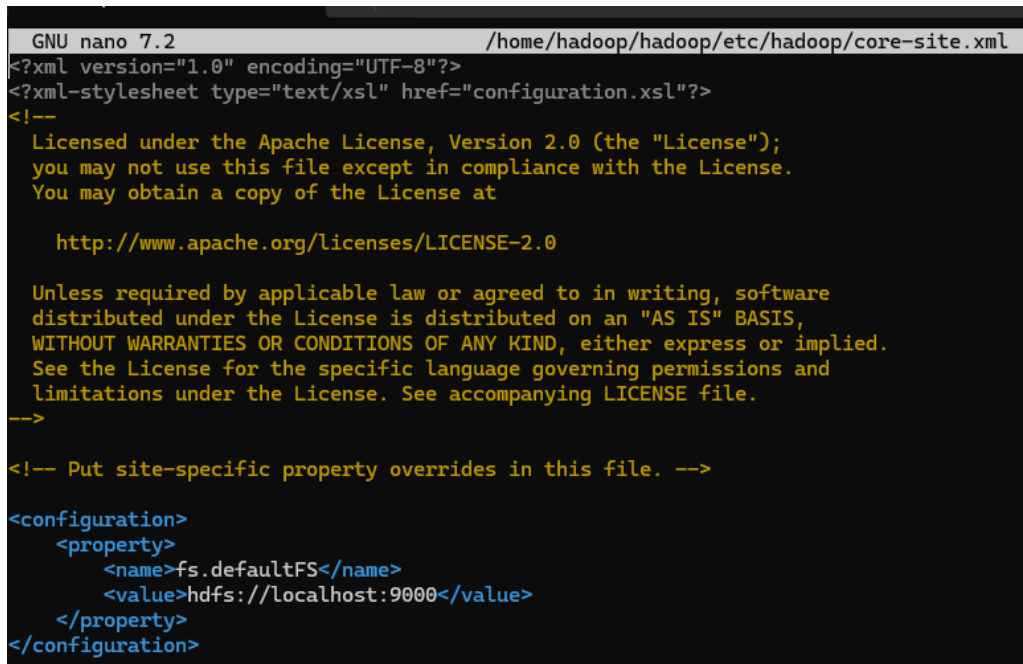
1.5.1 Step 2: Configure core-site.xml

1.5.2 Step 1: Set the Java path in hadoop-env.sh

core-site.xml informs the Hadoop daemon where NameNode runs in the cluster. It contains configuration settings of Hadoop core such as I/O settings that are common to HDFS MapReduce.

```
nano $HADOOP_HOME/etc/hadoop/core-site.xml
```

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```



```
GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/core-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

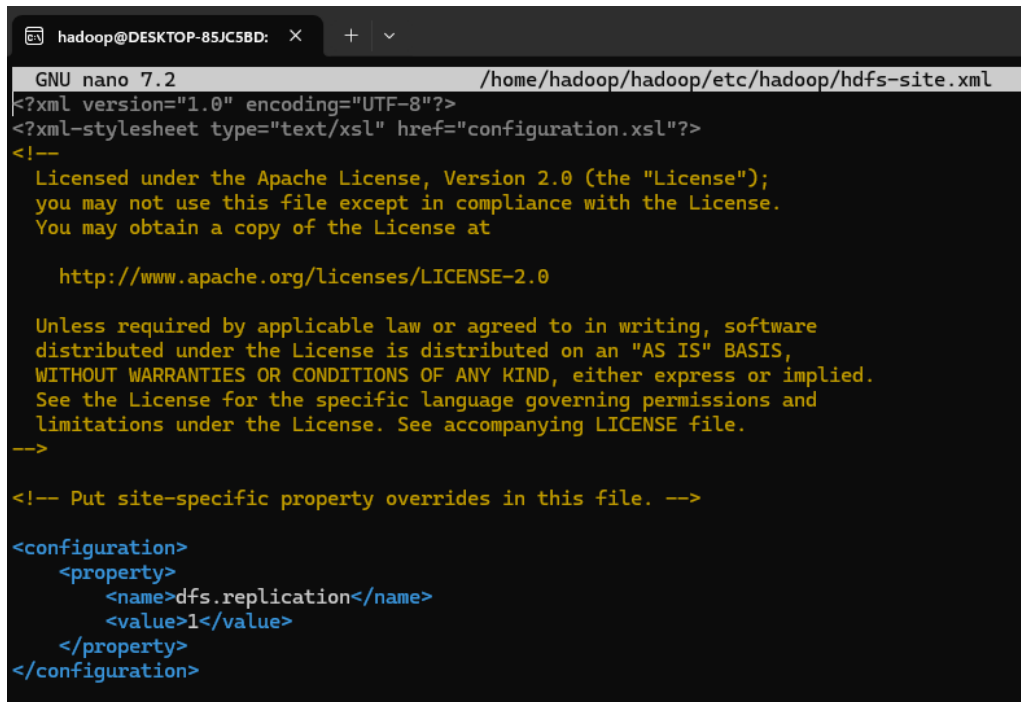
Figure 15: Configuration in core-site.xml.

1.5.3 Step 3: Configure hdfs-site.xml

```
nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
```

hdfs-site.xml contains configuration settings of HDFS daemons (i.e. NameNode, DataNode, Secondary NameNode). It also includes the replication factor and block size of HDFS.

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```



```
hadoop@DESKTOP-85JC5BD: X + v
GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/hdfs-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

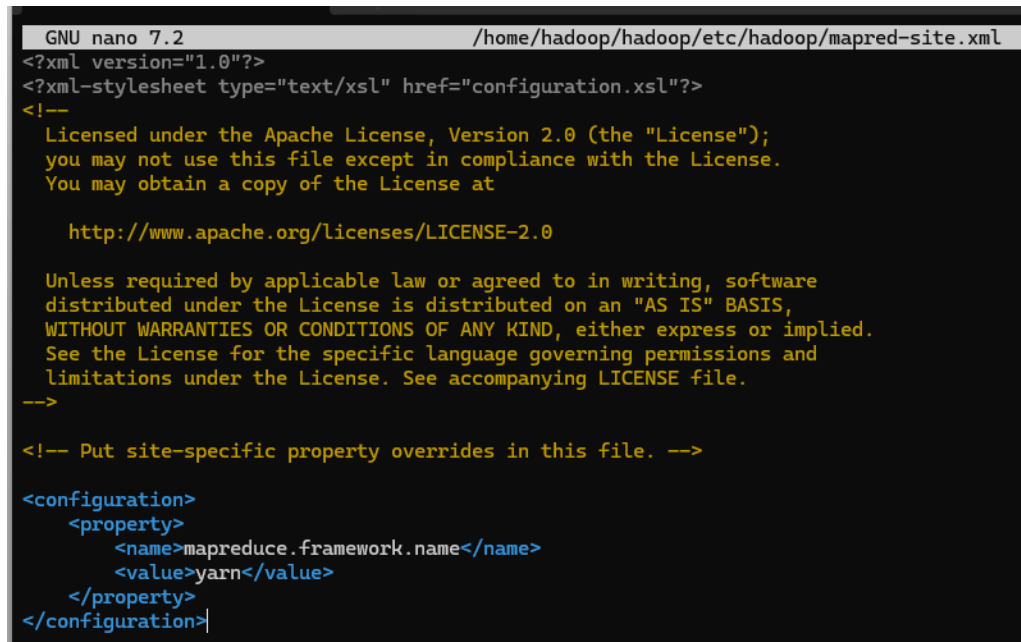
Figure 16: Setting dfs.replication to 1 in hdfs-site.xml.

1.5.4 Step 4: Configure mapred-site.xml

```
nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
```

mapred-site.xml contains configuration settings of MapReduce application like the number of JVM that can run in parallel, the size of the mapper and the reducer process, CPU cores available for a process, etc.

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```



```
GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/mapred-site.xml
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Figure 17: Selecting YARN as the MapReduce framework in mapred-site.xml.

1.5.5 Step 5: Configure yarn-site.xml

```
nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```

yarn-site.xml contains configuration settings of ResourceManager and NodeManager like application memory management size, the operation needed on program algorithm, etc.

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_HOME,HADOOP_COMMON_HOME,
      HADOOP_HDFS_HOME,HADOOP_CONF_DIR,
      CLASSPATH_PREPEND_DISTCACHE,...</value>
  </property>
</configuration>
```

```

GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/yarn-site.xml
<?xml version="1.0"?>
<!--
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License. See accompanying LICENSE file.
-->
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>

  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_COMMON_LIB_NATIVE_HOME</value>
  </property>
</configuration>

```

Figure 18: Configuring the YARN shuffle service and environment whitelist.

1.6 Formatting and Starting the Hadoop Cluster

1.6.1 Step 1: Format the NameNode

Now we go to the Hadoop home directory and format the Hadoop namenode before the first cluster startup:

```
hdfs namenode -format
```

```

hadoop@DESKTOP-85JC5BD:~$ hdfs namenode -format
WARNING: /home/hadoop/hadoop/logs does not exist. Creating.
2026-07-07 21:42:06,876 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG: host = DESKTOP-85JC5BD/127.0.1.1
STARTUP_MSG: args = [-format]
STARTUP_MSG: version = 3.5.0
STARTUP_MSG: classpath = /home/hadoop/hadoop/etc/hadoop:/home/hadoop/hadoop/share/hadoop/common/lib/javassist-3.30.2-GA.jar:/home/hadoop/hadoop/share/hadoop/common/lib/analyticsaccelerator-s3-1.3.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/curator-framework-5.2.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/HdrHistogram-2.1.12.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jsr305-3.0.2.jar:/home/hadoop/hadoop/share/hadoop/common/lib/kerby-config-2.0.3.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jakarta.annotation-api-1.3.5.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-cli-1.9.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-io-2.16.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/gson-2.9.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jakarta.activation-1.2.2.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-collections4-4.4.jar:/home/hadoop/hadoop/share/hadoop/common/lib/hadoop-tos-3.5.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/aopalliance-repackaged-2.6.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/j2objc-annotations-3.0.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/netty-transport-classes-epoll-4.1.130.Final.jar:/home/hadoop/hadoop/share/hadoop/common/lib/re2j-1.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-lang3-3.18.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/dnsjava-3.6.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jetty-security-9.4.58.v20250814.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jaxb-runtime-2.3.9.jar:/home/hadoop/hadoop/share/hadoop

```

Figure 19: Formatting the NameNode metadata for the new HDFS cluster.

1.6.2 Step 2: Start HDFS and verify its processes

```
start-dfs.sh
jps
```

```

hadoop@DESKTOP-85JC5BD:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [DESKTOP-85JC5BD]
DESKTOP-85JC5BD: Warning: Permanently added 'desktop-85jc5bd' (ED25519) to the list of known hosts.
hadoop@DESKTOP-85JC5BD:~$ jps
6112 DataNode
6359 SecondaryNameNode
6504 Jps
5983 NameNode

```

Figure 20: Starting HDFS and verifying NameNode, DataNode, and SecondaryNameNode.

- **Result:** The `jps` output contains **NameNode**, **DataNode**, and **SecondaryNameNode**, confirming that HDFS started successfully.

1.7 Cluster Status Verification via Web UI

1.7.1 Step 1: Explore the NameNode Web UI

The HDFS interface was opened at <http://localhost:9870>.

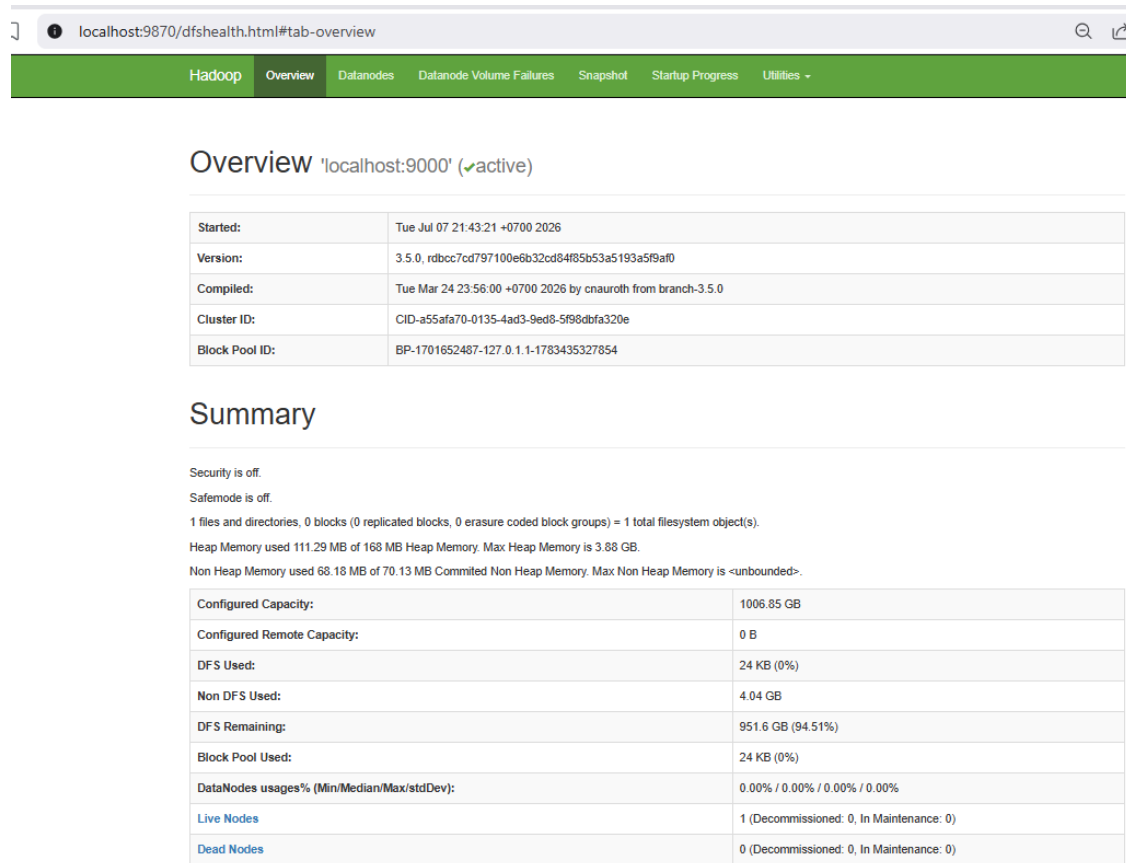


Figure 21: NameNode Web UI showing the active HDFS cluster.

- **State:** The NameNode is **active**, Hadoop version is **3.5.0**, and Safe Mode is off.
- **Storage:** The interface reports one live DataNode, zero dead nodes, and approximately 951.6 GB of DFS remaining capacity.

1.7.2 Step 2: Start YARN

```
start-yarn.sh
```

```
hadoop@DESKTOP-85JC5BD:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
```

Figure 22: Starting the ResourceManager and NodeManager.

1.7.3 Step 3: Explore the ResourceManager Web UI

The YARN interface was opened at <http://localhost:8088>.

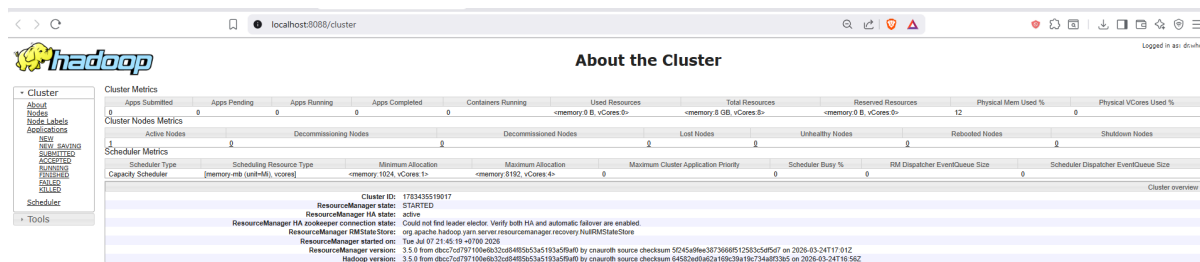


Figure 23: YARN ResourceManager Web UI showing one active node and available resources.

- **Cluster Metrics:** There is one active node and no running or completed applications before the Word Count submission.
- **Resources:** The ResourceManager reports approximately 8 GB of memory and 8 virtual cores.
- **Conclusion:** HDFS and YARN were both ready for the required file operations and MapReduce job.

2 Pseudo-Distributed Mode

2.1 Step 1: Creating the System Directory and Dedicated User

Create HDFS directory for experiment and create a required user :

```
hdfs dfs -mkdir -p /hcmus
sudo adduser khtn_23127096
```

```
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -mkdir -p /hcmus
hadoop@DESKTOP-85JC5BD:~$ sudo adduser khtn_23127096
[sudo] password for hadoop:
info: Adding user 'khtn_23127096' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group 'khtn_23127096' (1003) ...
info: Adding new user 'khtn_23127096' (1003) with group 'khtn_23127096 (1003)' ...
info: Creating home directory '/home/khtn_23127096' ...
info: Copying files from '/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for khtn_23127096
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] Y
info: Adding new user 'khtn_23127096' to supplemental / extra groups 'users' ...
info: Adding user 'khtn_23127096' to group 'users' ...
```

Figure 24: Creating /hcmus and the user khtn_23127096.

2.2 Step 2: Managing the Student Directory, Uploading Data, and Configuring Permissions

The student directory was created, a sample file was uploaded, and the owner and permissions were configured, these step aligned to the instructions of this lab 's definition:

```
hdfs dfs -mkdir -p /hcmus/23127096
echo "Hello Hadoop HDFS Lab 1 - 23127096" > ~/sample.txt
```

```
hdfs dfs -put -f ~/sample.txt /hcmus/23127096/
hdfs dfs -chown -R khtn_23127096:supergroup /hcmus/23127096
hdfs dfs -chmod 744 /hcmus/23127096
hdfs dfs -chmod 744 /hcmus/23127096/sample.txt
hdfs dfs -ls /hcmus
hdfs dfs -ls /hcmus/23127096
```

```
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -mkdir -p /hcmus/23127096
echo "Hello Hadoop HDFS Lab 1 - 23127096" > ~/sample.txt
hdfs dfs -put -f ~/sample.txt /hcmus/23127096/
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -chown -R khtn_23127096:supergroup /hcmus/23127096
hdfs dfs -chmod 744 /hcmus/23127096
hdfs dfs -chmod 744 /hcmus/23127096/sample.txt
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -ls /hcmus
hdfs dfs -ls /hcmus/23127096
Found 1 items
drwxr--r-- - khtn_23127096 supergroup          0 2026-07-07 22:21 /hcmus/23127096
Found 1 items
-rwxr--r-- 1 khtn_23127096 supergroup          35 2026-07-07 22:21 /hcmus/23127096/sample.txt
```

Figure 25: Uploading sample.txt, applying ownership and mode 744, and listing the HDFS paths.

The command `chown -R` changes the owner of the directory and all existing contents to `khtn_23127096`, with group `supergroup`. Permission mode `744` means that the owner has read, write, and execute permissions (`rwx`), while the group and other users only have read permission (`r-`). For the directory, execute permission allows traversal, while for the text file it marks the file as executable, although this permission is not normally required for a plain text file.

- **Directory result:** `drwxr--r-`, owner `khtn_23127096`.
- **File result:** `-rwxr--r-`, owner `khtn_23127096`.

2.3 Step 3: Executing the JAR Program for Automated Verification

The supplied JAR was executed from `/home/hadoop/run` using HDFS and my studentID directory:

```
mkdir -p ~/run
cd ~/run
java -jar ~/hadoop-test.jar 9000 /hcmus/23127096
cat 23127096_verification.txt
```

```
hadoop@DESKTOP-85JC5BD:~/run$ java -jar ~/hadoop-test.jar 9000 /hcmus/23127096
Trying to read /hcmus/23127096
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23127096/sample.txt
Your student ID: 23127096 (ensure it matches your student ID)
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
File written at /home/hadoop/run/23127096_verification.txt
hadoop@DESKTOP-85JC5BD:~/run$ cat 23127096_verification.txt
MAC=00-15-5D-67-CF-6D
b0f97255e30774ebaca5565b670646fb04b243ff6727aa820f6dd07f5072eeb1hadoop@DESKTOP-85JC5BD:~/run$ |
```

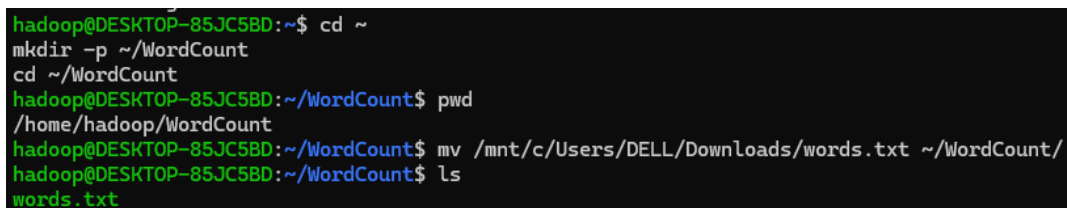
Figure 26: The verification JAR reading HDFS and generating 23127096_verification.txt.

3 Word Count

3.1 Step 1: Preparing the Local Workspace

A local workspace was created, and the supplied dictionary file was moved from the Windows Downloads directory into WSL(you can change the first directory corresponding to your file's path):

```
cd ~
mkdir -p ~/WordCount
cd ~/WordCount
pwd
mv /mnt/c/Users/DELL/Downloads/words.txt ~/WordCount/
ls
```

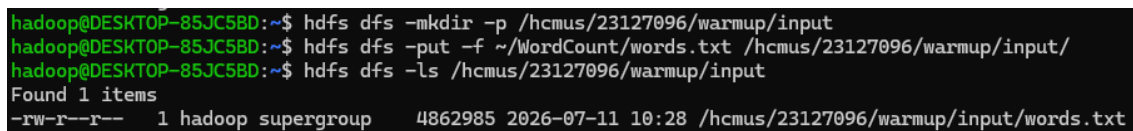
A terminal window showing the execution of commands to create a local workspace. The commands are: cd ~, mkdir -p ~/WordCount, cd ~/WordCount, pwd, mv /mnt/c/Users/DELL/Downloads/words.txt ~/WordCount/, and ls. The output shows the current directory is /home/hadoop/WordCount and the file words.txt has been moved there.

```
hadoop@DESKTOP-85JC5BD:~$ cd ~
hadoop@DESKTOP-85JC5BD:~$ mkdir -p ~/WordCount
hadoop@DESKTOP-85JC5BD:~$ cd ~/WordCount
hadoop@DESKTOP-85JC5BD:~/WordCount$ pwd
/home/hadoop/WordCount
hadoop@DESKTOP-85JC5BD:~/WordCount$ mv /mnt/c/Users/DELL/Downloads/words.txt ~/WordCount/
hadoop@DESKTOP-85JC5BD:~/WordCount$ ls
words.txt
```

Figure 27: Creating /home/hadoop/WordCount and moving words.txt into it.

3.2 Step 2: Creating the HDFS Input Directory and Uploading the Dataset

```
hdfs dfs -mkdir -p /hcmus/23127096/warmup/input
hdfs dfs -put -f ~/WordCount/words.txt \
    /hcmus/23127096/warmup/input/
hdfs dfs -ls /hcmus/23127096/warmup/input
```

A terminal window showing the execution of commands to create an HDFS directory and upload a file. The commands are: hdfs dfs -mkdir -p /hcmus/23127096/warmup/input, hdfs dfs -put -f ~/WordCount/words.txt /hcmus/23127096/warmup/input/, and hdfs dfs -ls /hcmus/23127096/warmup/input. The output shows the file has been uploaded to HDFS.

```
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -mkdir -p /hcmus/23127096/warmup/input
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -put -f ~/WordCount/words.txt /hcmus/23127096/warmup/input/
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -ls /hcmus/23127096/warmup/input
Found 1 items
-rw-r--r-- 1 hadoop supergroup 4862985 2026-07-11 10:28 /hcmus/23127096/warmup/input/words.txt
```

Figure 28: Uploading words.txt dataset to HDFS.

3.3 Step 3: Reading the Uploaded Dataset

The first ten records were displayed directly from HDFS:

```
hdfs dfs -cat /hcmus/23127096/warmup/input/words.txt | head
```

```
hadoop@DESKTOP-85JC5BD:~$ hdfs dfs -cat \
    /hcmus/23127096/warmup/input/words.txt \
    | head
2
1080
&c
10-point
10th
11-point
12-point
16-point
18-point
1st
cat: Unable to write to output stream.
```

Figure 29: Reading the first ten entries of `words.txt` from HDFS.

3.4 Step 4: Installing Scala and Creating the Source File

Scala and its runtime library were installed from the Ubuntu repository:

```
sudo apt install -y scala
```

```
hadoop@DESKTOP-85JC5BD:~$ sudo apt install -y scala
[sudo] password for hadoop:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libhawtjni-runtime-java libjansi-native-java libjansi1-java libjline2-java scala-library scala-parser-combinators
  scala-xml
Suggested packages:
  scala-doc
The following NEW packages will be installed:
  libhawtjni-runtime-java libjansi-native-java libjansi1-java libjline2-java scala scala-library
  scala-parser-combinators scala-xml
0 upgraded, 8 newly installed, 0 to remove and 0 not upgraded.
Need to get 25.1 MB of archives.
After this operation, 28.6 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 libhawtjni-runtime-java all 1.18-1 [28.9 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/universe amd64 libjansi-native-java all 1.8-2 [24.1 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/universe amd64 libjansi1-java all 1.18-3ubuntu1 [56.0 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/universe amd64 libjline2-java all 2.14.6-5 [150 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/universe amd64 scala-library all 2.11.12-5 [9586 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 scala-parser-combinators all 1.0.3-3.1 [365 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/universe amd64 scala-xml all 1.0.3-3.1 [615 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/universe amd64 scala all 2.11.12-5 [14.2 MB]
Fetched 25.1 MB in 4s (6131 kB/s)
```

Figure 30: Installing Scala and the required runtime packages.

Now we create a Scala file to run the Word Count program, the source code will later be submitted in my personal directory

```
cd ~
nano WordCount.scala
ls -l
```

```

hadoop@DESKTOP-85JC5BD:~$ nano WordCount.scala
hadoop@DESKTOP-85JC5BD:~$ ls -l
total 630024
drwxrwxr-x  2 hadoop hadoop      4096 Jul 11 10:17 WordCount
-rw-rw-r--  1 hadoop hadoop      5542 Jul 11 10:55 WordCount.scala
drwxr-xr-x 12 hadoop hadoop      4096 Jul  7 21:42 hadoop
-rw-rw-r--  1 hadoop hadoop 581280703 Apr  2 03:43 hadoop-3.5.0.tar.gz
-rwxrwxrwx  1 hadoop hadoop 63828099 Jul  7 22:01 hadoop-test.jar
drwxrwxr-x  2 hadoop hadoop      4096 Jul  7 22:32 run
-rw-rw-r--  1 hadoop hadoop        35 Jul  7 22:37 sample.txt

```

Figure 31: Creating /home/hadoop/WordCount.scala.

3.5 MapReduce Algorithm

3.5.1 Map Function

The mapper processes each input line and counts words whose first character belongs to the target set. Instead of using the letter itself as the intermediate key, the mapper uses the position of the letter in the `targetLetters` array.

Initialization:

Before processing the input records, the mapper's `setup` function emits one zero-valued pair for every target letter. The keys are the integer indices from 0 to 7.

These initial records ensure that the final output always contains exactly eight rows. If no word begins with a particular target letter, its final count will still be reported as zero.

Idea.

For each input line, the mapper uses a tokenizer to divide the line into words. For every non-empty word, its first character is converted to lowercase. The mapper then searches for this character in the `targetLetters` array.

If the character is found, its array index becomes the intermediate key and the value 1 is emitted. If the character is not in the target set, the word is ignored.

For example, "File", "Hadoop", and "science" produce the intermediate pairs (0, 1), (3, 1), and (7, 1), respectively.

Pseudo Code.

Algorithm Mapper initialization

```

1: function SETUP
2:   for index  $\leftarrow$  0 to 7 do
3:     EMIT(index, 0)
4:   end for
5: end function

```

Algorithm Map function

```

1: function MAP(key, value)
2:   line  $\leftarrow$  convert value to string
3:   words  $\leftarrow$  tokenize line
4:   for all word in words do
5:     if word is not empty then
6:       firstChar  $\leftarrow$  lowercase of the first character
7:       index  $\leftarrow$  position of firstChar in targetLetters
8:       if index  $\geq$  0 then
9:         EMIT(index, 1)
10:      end if
11:    end if
12:  end for
13: end function

```

3.5.2 Reduce Function

The reduce function receives an integer index and all partial counts associated with that index. Each index represents one target letter in the predefined array **f**, **i**, **t**, **h**, **c**, **m**, **u**, **s**.

Idea.

For each integer key, the reducer iterates through all associated values and adds them together. It then uses the key as an index into the **targetLetters** array to obtain the corresponding output letter.

For example, key 0 represents **f**, key 1 represents **i**, and key 7 represents **s**. After calculating the sum, the reducer immediately emits the corresponding letter and its total count.

The mapper uses integer indices as intermediate keys instead of letters. During the shuffle and sort phase, Hadoop sorts these keys in ascending numerical order. Therefore, the reducer receives them in the order 0, 1, 2, 3, 4, 5, 6, 7, which corresponds to **f**, **i**, **t**, **h**, **c**, **m**, **u**, **s**.

Since the program uses only one reducer, all results are written into one output file in the required order.

Pseudo Code.

Algorithm Reduce function

```

1: function REDUCE(index, values)
2:   sum  $\leftarrow$  0
3:   for all value in values do
4:     sum  $\leftarrow$  sum + value
5:   end for
6:   letter  $\leftarrow$  targetLetters[index]
7:   EMIT(letter, sum)
8: end function

```

3.5.3 Main Function

The driver validates the command-line arguments, configures the Hadoop job, and submits it for execution.

Idea.

The program requires exactly two arguments: the HDFS input path and the HDFS output path. If the number of arguments is incorrect, it displays a usage message and terminates.

The driver registers the mapper and reducer classes. The intermediate key and value types are both `IntWritable`, while the final output types are `Text` and `IntWritable`.

The number of reducers is explicitly set to one. This ensures that all eight results are written into a single output file. Together with the ascending order of the integer intermediate keys, this produces the required output order `f, i, t, h, c, m, u, s`.

Pseudo Code.

Algorithm Driver function

```

1: function MAIN(arguments)
2:   if number of arguments  $\neq$  2 then
3:     display the usage message
4:     terminate with error status
5:   end if
6:   create a Hadoop configuration
7:   create the Word Count job
8:   set the application JAR class
9:   set TokenizerMapper as the mapper
10:  set IntSumReducer as the reducer
11:  set map output types to integer key and integer value
12:  set final output types to text key and integer value
13:  set the number of reducers to 1
14:  set the input path from arguments[0]
15:  set the output path from arguments[1]
16:  submit the job and wait for completion
17: end function

```

3.5.4 MapReduce Data Flow

The complete data flow of the program can be summarized as follows:

words.txt $\longrightarrow$ Mapper setup and map $\longrightarrow$ Shuffle and sort by integer index $\longrightarrow$
 Single reducer $\longrightarrow$ part-r-00000

The mapper emits integer indices instead of letters. The mapping between each index and its target letter is shown below:

Index	Letter
0	f
1	i
2	t
3	h
4	c
5	m
6	u
7	s

The zero-valued records created by the mapper's setup phase do not change the counts. Their purpose is to ensure that every target letter appears in the final output, even when no matching word is found.

3.6 Step 5: Compiling the Scala Application

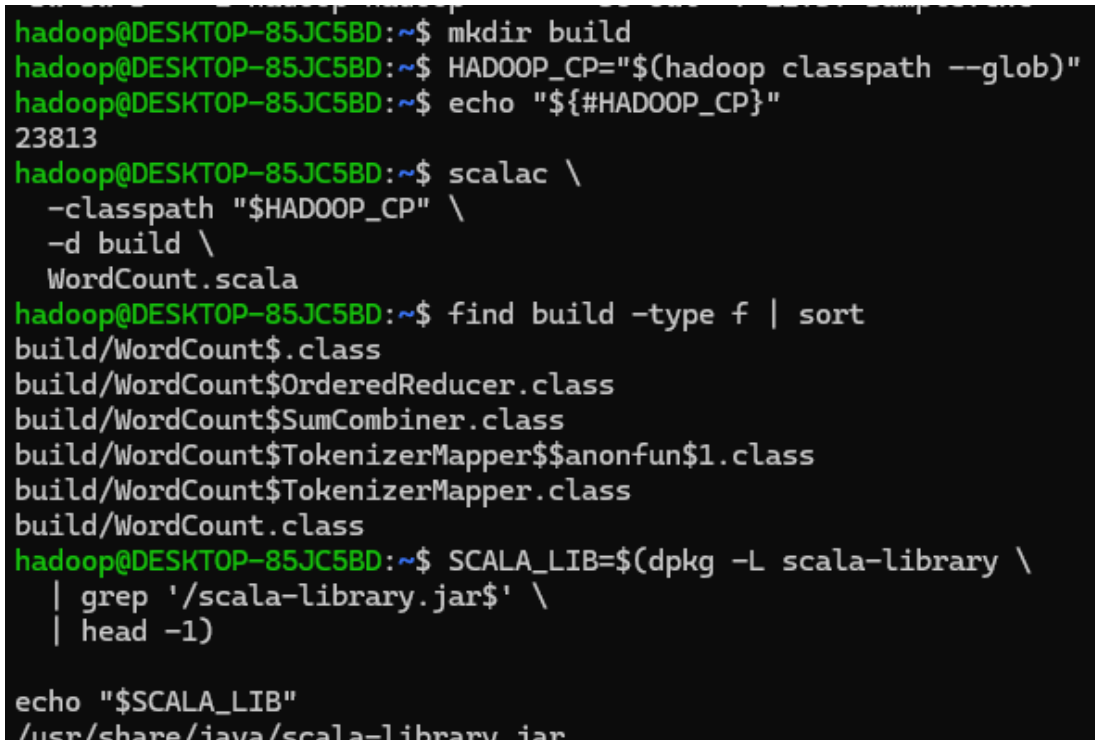
The full Hadoop classpath, which contains all necessary dependencies for the MapReduce framework, was supplied to the Scala compiler (`scalac`) via the `-classpath` option. Consequently, the source code was successfully compiled into Java bytecodes, and the resulting `.class` files were written to the target directory at `/home/hadoop/build`.

```
cd ~
mkdir build
HADOOP_CP="$(hadoop classpath --glob)"
echo "${#HADOOP_CP}"

scalac \
  -classpath "$HADOOP_CP" \
  -d build \
  WordCount.scala

find build -type f | sort

SCALA_LIB=$(dpkg -L scala-library \
  | grep '/scala-library.jar$' \
  | head -1)
echo "$SCALA_LIB"
```



```
hadoop@DESKTOP-85JC5BD:~$ mkdir build
hadoop@DESKTOP-85JC5BD:~$ HADOOP_CP="$(hadoop classpath --glob)"
hadoop@DESKTOP-85JC5BD:~$ echo "${#HADOOP_CP}"
23813
hadoop@DESKTOP-85JC5BD:~$ scalac \
  -classpath "$HADOOP_CP" \
  -d build \
  WordCount.scala
hadoop@DESKTOP-85JC5BD:~$ find build -type f | sort
build/WordCount$.class
build/WordCount$OrderedReducer.class
build/WordCount$SumCombiner.class
build/WordCount$TokenizerMapper$$anonfun$1.class
build/WordCount$TokenizerMapper.class
build/WordCount.class
hadoop@DESKTOP-85JC5BD:~$ SCALA_LIB=$(dpkg -L scala-library \
  | grep '/scala-library.jar$' \
  | head -1)
hadoop@DESKTOP-85JC5BD:~$ echo "$SCALA_LIB"
/usr/share/java/scala-library.jar
```

Figure 32: Compiling the Scala classes and locating `scala-library.jar`.

- **Result:** The driver, mapper, combiner, and reducer class files were generated successfully. The Scala runtime library was found at `/usr/share/java/scala-library.jar`.

3.7 Step 6: Packaging the JAR and Resolving Command Errors

To resolve missing dependency errors inside YARN containers, the Scala runtime was unpacked into the build directory and included in the application JAR. This produced a self-

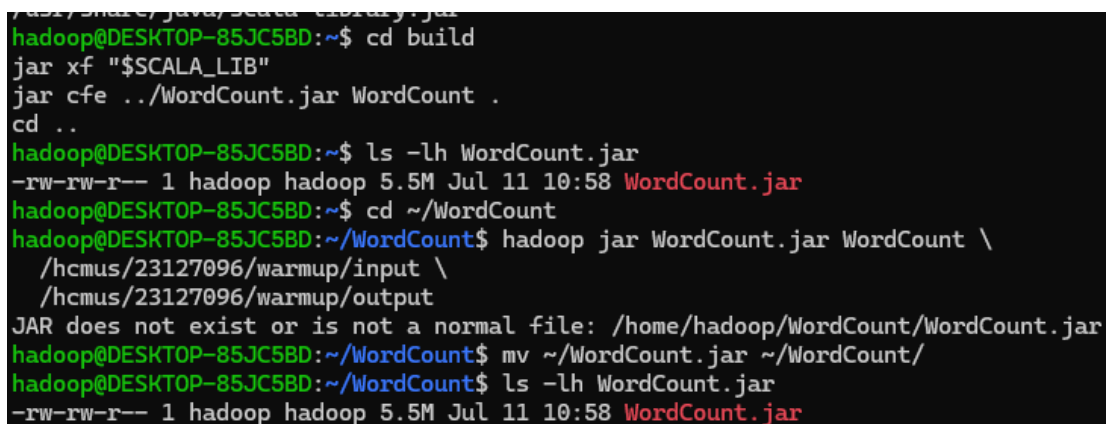
contained JAR for the YARN containers:

```
cd ~/build
jar xf "$SCALA_LIB"
jar cfe ../WordCount.jar WordCount .
cd ..
ls -lh WordCount.jar
```

The first execution attempt was made from `/home/hadoop/WordCount`, but the JAR had been created at `/home/hadoop/WordCount.jar`. Hadoop therefore reported that the expected JAR did not exist in the current directory, so I just moved it to the right place

```
cd ~/WordCount
hadoop jar WordCount.jar WordCount \
  /hcmus/23127096/warmup/input \
  /hcmus/23127096/warmup/output

# Resolution
mv ~/WordCount.jar ~/WordCount/
ls -lh WordCount.jar
```



```
hadoop@DESKTOP-85JC5BD:~$ cd build
jar xf "$SCALA_LIB"
jar cfe ../WordCount.jar WordCount .
cd ..
hadoop@DESKTOP-85JC5BD:~$ ls -lh WordCount.jar
-rw-rw-r-- 1 hadoop hadoop 5.5M Jul 11 10:58 WordCount.jar
hadoop@DESKTOP-85JC5BD:~$ cd ~/WordCount
hadoop@DESKTOP-85JC5BD:~/WordCount$ hadoop jar WordCount.jar WordCount \
  /hcmus/23127096/warmup/input \
  /hcmus/23127096/warmup/output
JAR does not exist or is not a normal file: /home/hadoop/WordCount/WordCount.jar
hadoop@DESKTOP-85JC5BD:~/WordCount$ mv ~/WordCount.jar ~/WordCount/
hadoop@DESKTOP-85JC5BD:~/WordCount$ ls -lh WordCount.jar
-rw-rw-r-- 1 hadoop hadoop 5.5M Jul 11 10:58 WordCount.jar
```

Figure 33: Creating the self-contained JAR, detecting the incorrect path, and moving it into `/WordCount`.

3.8 Step 7: Submitting the MapReduce Job to YARN

```
cd ~/WordCount
hadoop jar WordCount.jar \
  /hcmus/23127096/warmup/input \
  /hcmus/23127096/warmup/output
```

```

hadoop@DESKTOP-85JC5BD:~/WordCount$ hadoop jar WordCount.jar \
/hcmus/23127096/warmup/output
2026-07-11 11:02:11,979 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-07-11 11:02:12,235 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-07-11 11:02:12,241 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1783739604492_0001
2026-07-11 11:02:12,458 INFO input.FileInputFormat: Total input files to process : 1
2026-07-11 11:02:12,900 INFO mapreduce.JobSubmitter: number of splits:1
2026-07-11 11:02:12,634 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1783739604492_0001
2026-07-11 11:02:12,635 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-07-11 11:02:12,766 INFO conf.Configuration: resource-types.xml not found
2026-07-11 11:02:12,766 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-07-11 11:02:13,183 INFO impl.YarnClientImpl: Submitted application application_1783739604492_0001
2026-07-11 11:02:13,213 INFO mapreduce.Job: The url to track the job: http://DESKTOP-85JC5BD.localdomain:8088/proxy/application_1783739604492_0001/
2026-07-11 11:02:13,214 INFO mapreduce.Job: Running job: job_1783739604492_0001
2026-07-11 11:02:19,298 INFO mapreduce.Job: Job job_1783739604492_0001 running in uber mode : false
2026-07-11 11:02:19,299 INFO mapreduce.Job: map 0% reduce 0%
2026-07-11 11:02:23,345 INFO mapreduce.Job: map 100% reduce 0%
2026-07-11 11:02:28,301 INFO mapreduce.Job: map 100% reduce 100%
2026-07-11 11:02:28,391 INFO mapreduce.Job: Job job_1783739604492_0001 completed successfully
2026-07-11 11:02:28,449 INFO mapreduce.Job: Counters: 54
File System Counters
  FILE: Number of bytes read=86
  FILE: Number of bytes written=634565
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=4863109
  HDFS: Number of bytes written=64
  HDFS: Number of read operations=8
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0
Job Counters
  Launched map tasks=1
  Launched reduce tasks=1
  Data-local map tasks=1
  Total time spent by all maps in occupied slots (ms)=2521
  Total time spent by all reduces in occupied slots (ms)=2288
  Total time spent by all map tasks (ms)=2521
  Total time spent by all reduce tasks (ms)=2288
  Total vcore-milliseconds taken by all map tasks=2521
  Total vcore-milliseconds taken by all reduce tasks=2288
  Total megabyte-milliseconds taken by all map tasks=2681584
  Total megabyte-milliseconds taken by all reduce tasks=2342912
Map-Reduce Framework
  Map input records=466550
  Map output records=213514
  Map output bytes=1708112
  Map output materialized bytes=86
  Input split bytes=124

```

Figure 34: Successful completion of the MapReduce job with map and reduce at 100%.

3.9 Step 8: Exporting and Verifying the TSV Result

Now we read the result of the program above and then copy the content to the local "results.txt" file:

```

hdfs dfs -cat /hcmus/23127096/warmup/output/part-r-00000
rm -f results.txt
hdfs dfs -get \
/hcmus/23127096/warmup/output/part-r-00000 \
results.txt
cat results.txt

```

```

hadoop@DESKTOP-85JC5BD:~/WordCount$ rm -f results.txt
hadoop@DESKTOP-85JC5BD:~/WordCount$ hdfs dfs -get \
/hcmus/23127096/warmup/output/part-r-00000 \
results.txt
hadoop@DESKTOP-85JC5BD:~/WordCount$ cat results.txt
f      15870
i      15220
t      25223
h      18662
c      38934
m      25194
u      23790
s      50571

```

Figure 35: Reading the HDFS output and exporting it as the local TSV file results.txt.